Generalisable Learned Branching for Binary Linear Systems

Gwang-Jong Ko[a], Taesu Cheong[a,*], In-Chan Choi[a,*]

[a]School of Industrial and Management Engineering, Korea University, 145, Anam-ro, Seongbuk-gu, Seoul, 02841, Republic of Korea

Abstract

A binary linear system (BLS) asks for a vector $\mathbf{x} \in \{0,1\}^n$ with $A\mathbf{x} = \mathbf{b}$ for a 0/1 matrix A; we study the solution-finding version, in which a solution exists and has to be produced. It has no objective, its linear relaxation is wide and uninformative, and unit propagation is inert near the root, so a complete depth-first search is decided by its branching choices. Learned branching has been developed for mixed-integer programs and for SAT, and we show that each of its three main lines loses something when transplanted to a BLS: imitation of strong branching degenerates to LP probing because there is no bound to improve; message-passing marginals cost two orders of magnitude more than a network forward pass per node on a dense factor graph; and reinforcement learning from scratch has to discover a value ordering that the problem provides exactly. We present a branching rule learned in two stages from two exact signals. Stage 1 learns *which value* to try. On enumerable instances the conditional marginal of each free variable is exact, and because conditioning a BLS on a partial assignment is the same as reducing it, one graph network trained on reduced instances predicts at every node. Stage 2 learns *which variable* to branch on. The number of nodes a decision's subtree expands is produced exactly by the search, and an actor-critic fine-tuning on that cost, started at the Stage-1 rule with the network frozen, changes the branching order alone. We evaluate against seven classical rules (most-integral and most-fractional LP branching, maxSD counting, dom/wdeg, impact, full LP probing, uniform) and two reference signals (exact marginals and belief propagation) in one search loop, on 100-instance sets of three sizes. The learned rule expands fewer nodes than every single-relaxation rule at every size (1.5–3.2× against the strongest of them, sign test $p \le 0.035$) and, at 28×80, is also faster than all of them (2.0–3.1×, $p \le 0.002$). Only LP probing expands fewer nodes, at 5–8× the time. Trained at $n \le 60$, the rule keeps its advantage at $n = 70$ and 80, and the advantage grows with size. A value-by-variable cross ablation shows that the gain lies in the variable choice: the cost-tuned variable with the LP rounding value matches the full rule, while the LP variable with the exact-marginal value is 28× worse. Plugged into SCIP as a branching rule the policy expands 1.9–3.8× fewer nodes than reliability pseudocost branching with conflict analysis on or off and matches its time at the two larger sizes, while a complete solver with presolve and clause learning remains faster end to end; the gap is per-node cost, not tree size.

Keywords: Binary linear systems, Learning to branch, Exact conditional marginals, Reinforcement learning, Depth-first search, Graph neural networks

1. Introduction

A binary linear system (BLS) is the solution-finding problem

$$\text{find } \mathbf{x} \in \{0,1\}^n \text{ such that } A\mathbf{x} = \mathbf{b}, \qquad A \in \{0,1\}^{m\times n},\ \mathbf{b} \in \mathbb{Z}^m_{\ge 0}, \tag{1}$$

with solution set $\mathcal{S}$. It is the general form of a reconstruction task in laser-based non-destructive testing: a binary occupancy image must reproduce a set of integer projections exactly, the scanned object exists, so a solution exists and is often not unique, and the reconstruction has to keep pace with a production line. Throughout, an instance always has a solution and the task is to produce one; deciding whether a system is infeasible is outside the scope of this paper (Section 3.1). The instances that matter have right-hand sides near the middle of each row's range, and they are hard out of proportion to their size.

Three properties defeat the standard tools on such instances. There is no objective, so the bound-based pruning of branch-and-bound has nothing to work with. The linear relaxation is always feasible but carries little information. The relaxation polytope $\{\mathbf{x} \in [0,1]^n : A\mathbf{x} = \mathbf{b}\}$ has dimension $n - \operatorname{rank} A \approx 0.65\,n$ and contains points near $\frac{1}{2}\mathbf{1}$, because every right-hand side sits at the middle of its row's range. A vertex returned by the simplex has at most m fractional coordinates, but which of the remaining $n - m$ coordinates sit at 0 or 1 is decided by the pivoting path rather than by $\mathcal{S}$: rounding the root vertex agrees with a solution on 59% of the variables, against 50% for chance and 69% for the Bayes ceiling of Section 3.3. Unit propagation, the workhorse of constraint solvers, fires only when a row is saturated, which never happens at the root and rarely near it. Lattice basis reduction with enumeration, the classical method for this family, is sound and solves every instance we study, at a cost that grows faster with size than a complete search's; what decides a complete search, on the instances the lattice leaves and at the sizes where it stops, is the quality of its branching decisions, and that is the object of this paper.

*Corresponding authors.

Email addresses: tcheong@korea.ac.kr (Taesu Cheong), ichoi@korea.ac.kr (In-Chan Choi)

1.1. What learned branching leaves open here

Three lines of work learn branching decisions, and Section 2 reviews them: imitation of strong branching, approximate marginals by message passing, and reinforcement learning on subtree size. Each was developed for a problem with an objective, a sparse constraint graph or a sparse reward, and each loses something when applied to a BLS. Without an objective, strong branching has no bound improvement to score and degenerates to LP probing, a rule that pays one relaxation per free variable per node. On the dense factor graph of a BLS, belief propagation needs heavy damping and hundreds of iterations per estimate, and its root-level use does not supply the conditional marginals a complete search needs at every node. Reinforcement learning from scratch has to discover which value to try first from the same sparse signal that teaches it which variable to branch on, although a BLS makes the optimal first value available exactly on the instances small enough to enumerate. What is missing is a branching rule that takes the exact signals a BLS offers, the conditional marginal of a free variable and the exact cost of a branching decision, learns the value and the variable from them separately, and does so once, at a size where the signals are affordable, for deployment at sizes where they are not. This paper presents such a framework (Section 4) and evaluates it against the classical rules and against the three transplants in one search loop.

1.2. Research questions

The experiments are organised around four questions, each answered on 100-instance sets of three sizes, with every branching rule run in the same search loop.

RQ1 Does the learned rule beat classical branching rules and the three transplanted learned-branching baselines, in nodes *and* in time? Since a smaller tree buys time only if the per-node cost allows, the question includes where that cost goes.

RQ2 Does a rule trained at $n \leq 60$ generalise to larger instances, and does its advantage shrink or grow with size?

RQ3 Are the two signals interchangeable, and is each stage necessary? This includes what the cost-tuned policy actually changed in the search.

RQ4 How does the rule fare against complete solvers, as a branching rule inside one and end to end?

1.3. Contributions

1. A learned branching framework for BLS that learns the two parts of a branching decision from two exact signals of the problem, with the value learned where it is enumerable and carried to every node of a complete search by a reduction identity, and the branching order learned from exact subtree costs with the value model frozen, so that the gain is attributable and transfers to sizes the model never saw.
2. A controlled answer to RQ1–RQ4: seven classical rules, two reference signals and three transplanted learned baselines in one loop; 100-instance sets at three sizes; a value-by-variable cross ablation, training-variant ablations and a refutation-structure analysis of what the cost signal taught; the rule as a SCIP branching plug-in against reliability pseudocost branching; and end-to-end comparisons with SCIP, CP-SAT, Gurobi and lattice enumeration.
3. Two negative findings, reported as such: the exact-marginal value adds nothing over LP rounding once the variable choice is learned, and the learned rule does not close the wall-clock gap to a complete solver.

Section 2 reviews related work, Section 3 states the problem and the reduction identity, Section 4 the method, Section 5 the experiments by research question, and Section 6 concludes with limitations.

2. Related work

2.1. Marginals of the solution set as a branching signal

Branching on the fraction of solutions that agree with a decision predates learning: counting-based search (Pesant et al., 2012) branches on the variable–value pair of maximum solution density computed inside individual constraints, and belief-propagation-guided decimation (Montanari et al., 2007) fixes the most biased variable under approximate marginals. NSNet (Li and Si, 2022) trains a graph network on exact marginals of the satisfying assignments of a SAT formula, obtained by enumerating them, and rounds the estimates into an initial assignment for local search; MIP-GNN (Khalil et al., 2022), Neural Diving (Nair et al., 2020) and predict-and-search (Han et al., 2023) predict per-variable biases from collected near-optimal solutions and use them to fix variables, warm-start or restrict a search. In all of these the marginal or bias is estimated once, at the root, and used to fix variables or to seed an incomplete search; none supplies a marginal conditional on the partial assignment at every node of a complete search, which is what a branching rule consumes.

2.2. Learning to branch

Khalil et al. (2016) and Gasse et al. (2019) imitate strong branching, the latter with the bipartite variable–constraint graph network we also use; imitation targets an expert's ranking rather than the tree size the ranking is meant to reduce. Optimising tree size directly is the subject of a line of reinforcement-learning work (surveyed, with the wider use of reinforcement learning inside exact solvers, by Mazyavkina et al., 2021): FMSTS (Etheve et al., 2020) uses the size of a node's subtree as an observable Q-value, shows that under depth-first search minimising every subtree minimises the whole tree, and learns from scratch; tree MDPs (Scavuzzo et al., 2022) formalise the per-subtree credit assignment; retro branching (Parsonson et al., 2023) learns from retrospectively extracted subtree trajectories; SORREL (Feng and Yang, 2025) initialises from demonstrations. Neuro# (Vaezipoor et al., 2021) trains a graph network

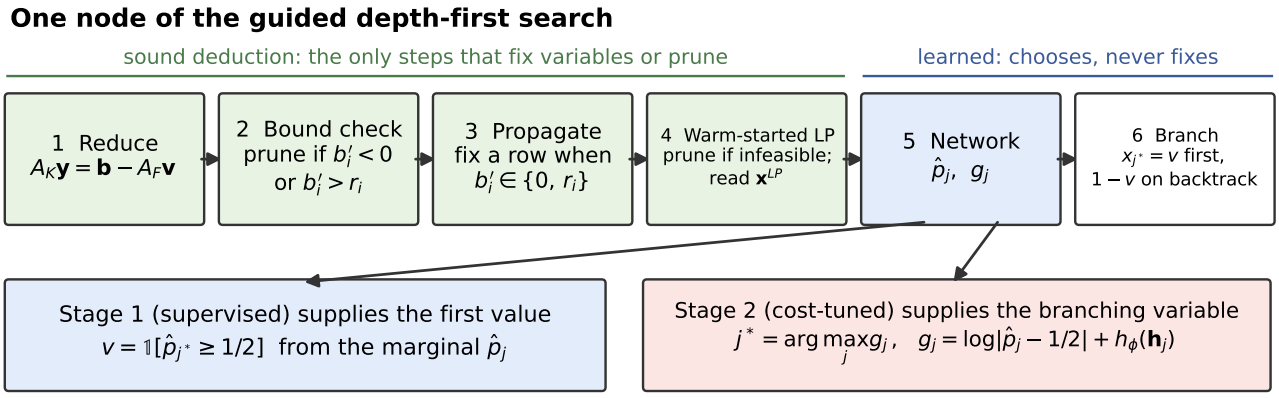


Figure 1: The guided depth-first search. Inside a node, steps 1–4 are sound and are the only steps that fix variables or prune; the network at step 5 only chooses, the value from Stage 1 and the variable from Stage 2. Lattice reduction, when used, runs before the search as a preprocessing step and is switched off in every branching comparison.

on the residual formula of a #SAT solver by evolution strategies and generalises to larger instances; in the operations-research literature, learned policy and value networks have likewise been placed inside a heuristic tree search (Hottung et al., 2020), and graph networks over the constraint structure of a problem have become the standard representation for such learned heuristics (Smit et al., 2025). The classical statements of the objective these methods optimise are the fail-first principle (Haralick and Elliott, 1980) and impact-based search (Refalo, 2004); for mixed-integer programs in which feasibility rather than the bound is the difficulty, Pryor and Chinneck (2011) showed that branching rules designed to force change in the relaxation reach integer feasibility faster than bound-driven rules, which is the regime a BLS is in throughout. Value ordering has been learned in constraint programming from solutions (Chu and Stuckey, 2015) and by deep Q-learning (Cappart et al., 2021), in each case jointly with or independently of the variable choice.

2.3. *Market split, lattice methods and complete solvers*

The market split problem of Cornuéjols and Dawande (1999) is the benchmark that exposes the weakness of branch-and-bound on systems like (1): m equality rows over $n = 10(m-1)$ binary variables, coefficients drawn uniformly from $\{0, \ldots, 99\}$, and each right-hand side set to half its row sum, so that the relaxation polytope is wide, its vertices far from integrality, and bounding prunes almost nothing; branch-and-cut stops at $m \approx 7$. Aardal et al. (2000b,a) solved these instances by reformulating them on a lattice: the Aardal–Hurkens–Lenstra embedding turns a solution into one of the shortest vectors of a lattice, LLL/BKZ reduction (Lenstra et al., 1982; Schnorr and Euchner, 1994) finds it, and the recovered vector is verified, so the method is sound. Lattice enumeration remains the state of the art on the benchmark: Wassermann (2025) solves QOBLIB market-split instances up to $m = 14$ on one CPU. Our instances share the ingredients that make market split hard, right-hand sides at the middle of their range and a wide relaxation polytope, and differ in having 0/1 coefficients, a planted solution and a search regime of up to 80 variables; lattice reduction is therefore a natural preprocessing stage for a BLS solver. Constraint solvers with clause learning (Marques-Silva and Sakallah, 1999; Ohrimenko et al., 2009) manufacture information as they descend and set the wall-clock reference we compare against. Direct satisfiability prediction has a weak record (Selsam et al., 2019; Li et al., 2023), and message-passing networks provably cannot separate some feasible/infeasible pairs (Chen et al., 2023). Learning a per-node estimate that replaces an expensive per-node computation is an instance of amortised inference (Amos, 2023; Gershman and Goodman, 2014).

2.4. *What these approaches leave open for a BLS*

The three learned-branching families closest to our setting each lose something essential when transplanted to a feasibility system. *Imitation of strong branching* (Khalil et al., 2016; Gasse et al., 2019) presupposes an objective: the expert scores a candidate by the bound improvement of its two children, and a BLS has no bound to improve. The only information strong branching retains on a feasibility system is whether a child's relaxation is infeasible, so the expert degenerates to LP probing, a rule that fixes what it can refute and then branches on the LP value, at one relaxation per free variable per node; imitating it would inherit its ranking without its per-node cost, and whether a learned imitation keeps the expert's tree is a question we leave open (Section 6); what we test is the expert itself. *Approximate marginals by message passing* (Li and Si, 2022; Montanari et al., 2007) are well defined for a BLS: each row is a cardinality factor, and sum-product with exact factor messages is implementable. Two things go wrong. The factor graph is dense, every row touching about a third of the variables, so loopy belief propagation needs heavy damping and hundreds of iterations where it converges; and NSNet-style use estimates marginals once at the root and decodes an assignment for an incomplete

local search, whereas a complete search needs marginals *conditional* on the partial assignment at every node. *Reinforcement learning from scratch on subtree size* (Etheve et al., 2020; Scavuzzo et al., 2022) supplies the right credit assignment for a depth-first search. What it does not supply is the value choice: a single policy over variable–value pairs must learn which value to try first from the same sparse reward that teaches it which variable to branch on, while a BLS makes the optimal value available exactly, as a conditional marginal, on the instances it can be enumerated on; nor does it say where a learned order should start from or how a gain in tree size should be attributed. Two further gaps are not about learning. Lattice reduction with bounded enumeration (Aardal et al., 2000a; Wassermann, 2025) is sound and solves every instance we study, in a fraction of a second at $n = 60$ and in minutes at $n = 80$ (Section 5.5); it is a natural preprocessing step for a BLS solver, and a comparison of branching rules has to be made on the search alone, with such a stage switched off. And a network cannot be trusted with a verdict on feasibility (Selsam et al., 2019; Chen et al., 2023), so a learned rule must only order the search and leave it complete. The framework of Section 4 is built to close the three learning gaps under the two constraints: it learns the value from the exact conditional marginals the problem offers and the variable from the exact subtree costs the search produces, separately and in that order, and it never fixes a variable or prunes.

3. Problem statement and preliminaries

3.1. Instances and scope

We study feasible instances of (1): every instance is generated by planting a solution, so $\mathcal{S} \neq \emptyset$, and deciding infeasibility is outside the scope of this paper. The construction is the market-split recipe of Section 2 with 0/1 coefficients. The generator draws a random A in which each entry is 1 with probability 0.3, plants a random $\mathbf{x}^*$ with exactly $n/2$ ones and sets $\mathbf{b} = A\mathbf{x}^*$, so every right-hand side sits at the middle of its row's range, where the number of 0/1 completions is largest and propagation cannot fire. Among 20 candidate plantings per A it keeps the one whose relaxation polytope is widest, measured by the mean distance between the vertices returned by four random linear objectives; the relaxation of the kept instance is therefore as uninformative as the generator can make it. Uniqueness is not imposed; the median number of solutions, where enumeration completes, is 64, 112 and 113 at $n = 25$, 50 and 60.

3.2. Conditioning is reduction

A node of a depth-first search over (1) is a partial assignment $\mathbf{x}_F = \mathbf{v}$ on a set F of fixed variables, with K the free ones. Let $\mathcal{S}(F, \mathbf{v}) = \{\mathbf{x} \in \mathcal{S} : \mathbf{x}_F = \mathbf{v}\}$ be the solutions consistent with the node and

$$p_j = \Pr[x_j{=}1 \mid A, \mathbf{b}, \mathbf{x}_F{=}\mathbf{v}] = \frac{|\{\mathbf{x} \in \mathcal{S}(F, \mathbf{v}) : x_j{=}1\}|}{|\mathcal{S}(F, \mathbf{v})|}, \quad j \in K, \tag{2}$$

the conditional marginal, defined by the uniform measure on $\mathcal{S}$ (the planted generator does not induce this measure; it is the definition of the target).

Proposition 1. *With A_F, A_K the column submatrices indexed by F, K, the map $\mathbf{x} \mapsto \mathbf{x}_K$ is a bijection from $\mathcal{S}(F, \mathbf{v})$ onto $\{\mathbf{y} \in \{0,1\}^{|K|} : A_K\mathbf{y} = \mathbf{b} - A_F\mathbf{v}\}$. Hence the conditional marginals of the original instance are the ordinary marginals of the reduced instance.*

Proof. $A\mathbf{x} = A_F\mathbf{x}_F + A_K\mathbf{x}_K$; with $\mathbf{x}_F = \mathbf{v}$ fixed, $A\mathbf{x} = \mathbf{b}$ iff $A_K\mathbf{x}_K = \mathbf{b} - A_F\mathbf{v}$, and $\mathbf{x}_F$ is determined, so the map is a bijection; the marginal statement follows by counting. □

Two consequences (Figure 2). First, one architecture, one label generator and one training loop serve every node: a depth-d training state is stored as an instance on $n - d$ variables, and a network trained on small instances is evaluated on the reduced instances that a search on a larger one produces. Second, $p_j \in \{0, 1\}$ means every surviving solution agrees on x_j, and branching the other way empties the subtree. We call such variables FORCED and the rest OPEN: either branch of a OPEN variable still reaches a solution, so a wrong first value costs nothing beyond exploring a subtree that contains one. Only FORCED errors cost a backtrack.

3.3. What a node looks like, and the accuracy ceiling

Because $\mathcal{S}(F, \mathbf{v})$ can be enumerated on small instances, the Bayes-optimal per-variable accuracy of any predictor of a solution is computable,

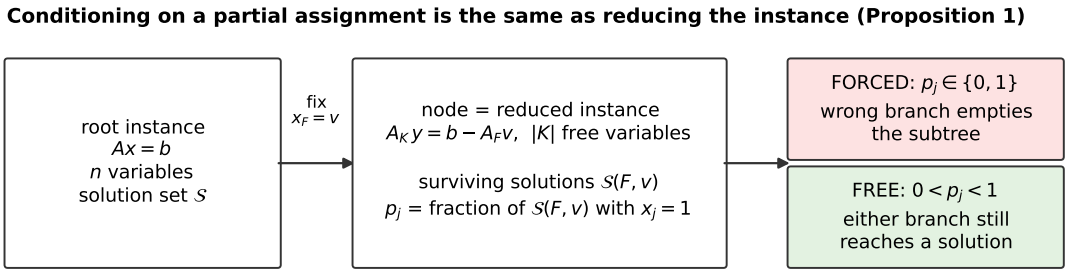
$$\mathrm{ceil}(F, \mathbf{v}) = \frac{1}{|K|} \sum_{j \in K} \max(p_j, 1 - p_j), \tag{3}$$

attained by answering the majority value of the surviving solutions. On held-out in-tree states of 10×25 instances the surviving solution set shrinks from a median of 74 at the root to a singleton by depth 10, and the FORCED share of the free variables rises from 3% at the root to 41% at depth 5, 73% at depth 8 and 94% at depth 12. At the root almost no decision can cost anything and the ceiling (3) is dominated by OPEN variables; from depth 6 on, most decisions can. Figure 3 shows the Stage-1 network against that ceiling by depth. This is the regime a branching heuristic must be good in, and it is where prediction is trained and evaluated below.

4. Proposed method

4.1. Overview

The solver is a depth-first search whose branching decisions are guided by a learned network (Figure 1); nothing learned ever fixes a variable or prunes, so the search stays complete. The network (Figure 4) is trained in two stages (Figure 5): Stage 1 fits its marginal head to exact conditional marginals of enumerated instances, Stage 2 adds a policy head on the frozen trunk and fine-tunes it on the exact number of nodes that every branching decision costs. Lattice reduction in the Aardal–Hurkens–Lenstra embedding (Aardal et al., 2000b), with bounded enumeration after BKZ, is a sound preprocessing step that solves every instance we study, cheaply at small sizes (Section 5.5); it is not part of the method and is switched off in every comparison of branching rules, so that the measured difference is the branching decision and nothing else.



Only errors on FORCED variables cost a backtrack. Their share is 3% at the root, 41% at depth 5 and 73% at depth 8 (10×25 in-tree states, $p = 0.3$), so the network is trained on reduced instances (in-tree states); labels are exact because $\mathcal{S}$ is enumerated.

Figure 2: Proposition 1 and the partition it induces. A node with $\mathbf{x}_F = \mathbf{v}$ is the smaller BLS $A_K\mathbf{y} = \mathbf{b} - A_F\mathbf{v}$; its conditional marginals are the ordinary marginals of that instance. Variables on which every surviving solution agrees are FORCED; only branching against them costs a backtrack.

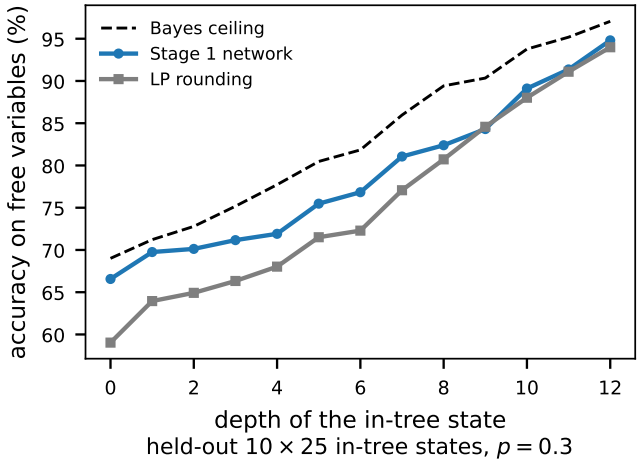


Figure 3: Prediction against the exact marginals on held-out in-tree states of 10×25 instances, by depth: per-variable accuracy of the Stage-1 network and of relaxation rounding against the Bayes ceiling (3).

4.2. *Guided depth-first search*

Every instance is searched from scratch by the depth-first procedure of Algorithm 1. A node is a partial assignment; by Proposition 1 it is handled as the reduced instance $(A', \mathbf{b}') = (A_K, \mathbf{b} - A_F\mathbf{v})$, and r_i denotes the number of free variables in row i of A'.

Three design choices matter for what follows. Propagation on a 0/1 row reads "exactly b'_i of the r_i free variables in this row are 1" and can fix variables only when $b'_i \in \{0, r_i\}$, which is why it is inert near the root of our instances. The relaxation is not rebuilt per node: one linear program over the original n variables is kept for the whole search and a node is expressed as a set of bound changes, so consecutive nodes, a child or a sibling after backtracking, differ by a few bounds and the dual simplex resumes from the parent basis; this is 50× cheaper than rebuilding and is what lets the loop process about 1,000 nodes per second in Python. Its infeasibility test is strictly stronger than propagation, since it sees linear combinations of rows, and it is the last sound step before the network. The vertex $\mathbf{x}^{LP}$ it returns is the main input feature of the network.

Table 1: Algorithm 1: one node of the guided depth-first search. Steps 1–4 are sound and are the only steps that fix variables or prune; the learned components enter only at step 5.

1	**Reduce.** Pop $(F, \mathbf{v})$; form $A' = A_K$, $\mathbf{b}' = \mathbf{b} - A_F\mathbf{v}$. Drop rows with $r_i = 0$; if none remain, verify $A\mathbf{x} = \mathbf{b}$ and return $\mathbf{x}$.
2	**Bound check.** If $b'_i < 0$ or $b'_i > r_i$ for some row, discard the node.
3	**Propagate.** While some row has $b'_i = 0$ (fix its free variables to 0) or $b'_i = r_i$ (fix them to 1): apply the fixings; on a contradiction discard the node; if variables were fixed, push the reduced node and return.
4	**Relax.** Re-optimise the single warm-started relaxation $\{A\mathbf{x} = \mathbf{b},\ 0 \le \mathbf{x} \le 1\}$ with the bounds of F tightened to $\mathbf{v}$ (dual simplex from the previous basis, 0.08 ms). If infeasible, discard the node; else read the vertex $\mathbf{x}^{LP}_K$.
5	**Predict.** One forward pass of the network on the bipartite graph of $(A', \mathbf{b}', \mathbf{x}^{LP}_K)$ gives $\hat{p}_j$ and g_j for $j \in K$ (0.3–0.5 ms on the dense path of Section 4.5). Choose $j^\star = \arg\max_j g_j$ and $v = \mathbb{1}[\hat{p}_{j^\star} \ge \frac{1}{2}]$.
6	**Branch.** Push the child $x_{j^\star} = 1 - v$, then the child $x_{j^\star} = v$, so that v is explored first and $1 - v$ on backtracking.

4.3. *Stage 1: exact conditional marginals as the target*

4.3.1. *Network*

The network f_θ (Figure 4) is a bipartite variable–constraint graph network with graph attention (Veličković et al., 2018), the architecture family of Gasse et al. (2019). Each variable node carries three features (its relaxation value x^{LP}_j, its column density, its fractionality $|x^{LP}_j - \text{round}(x^{LP}_j)|$), each constraint node three (its right-hand side b'_i/r_i, its density $r_i/|K|$, its relaxation residual $(b'_i - (A'\mathbf{x}^{LP})_i)/r_i$), and an edge joins j to i where

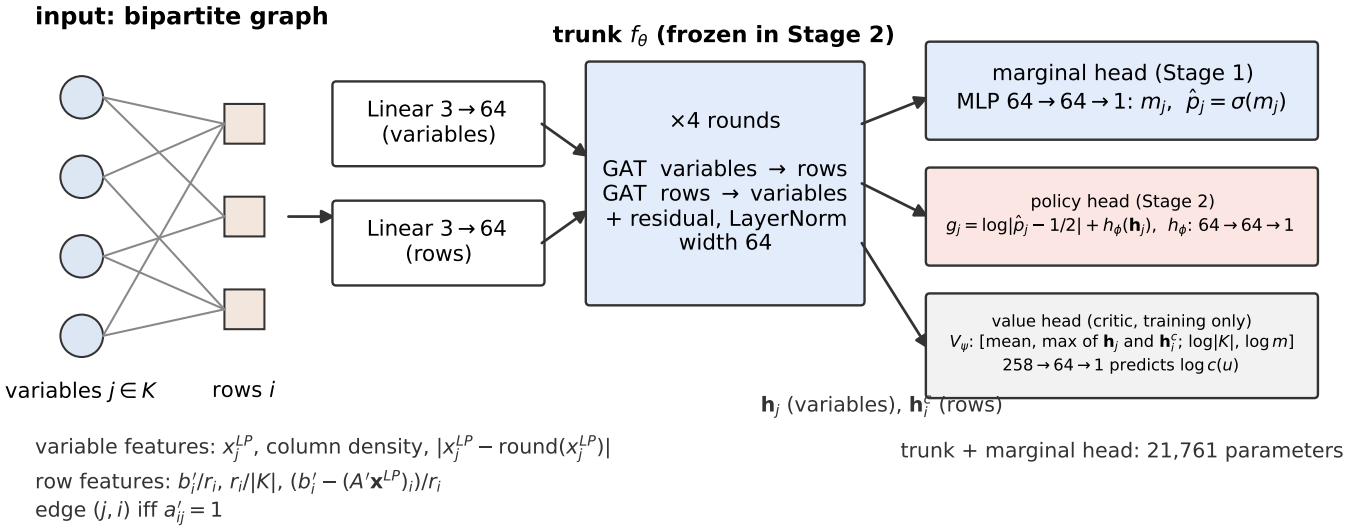


Figure 4: The network. The reduced instance enters as a bipartite graph with three features per variable and per row; a trunk of four graph-attention rounds produces an embedding per variable and per row; three heads read them. The marginal head is trained in Stage 1, the policy head in Stage 2 on the frozen trunk, and the value head serves only as the critic during Stage 2.

$a'_{ij} = 1$. Linear layers lift both to width 64; four rounds of attention then alternate variable-to-constraint and constraint-to-variable messages with residual connections and layer normalisation, and a two-layer marginal head maps each variable embedding $\mathbf{h}_j$ to one logit, $\hat{p}_j = \sigma(m_j)$; 21,761 parameters in all. The two heads added in Stage 2 are described in Section 4.4. The architecture is standard; the contribution is the target.

4.3.2. Target and training states

On instances small enough to enumerate (10×25), $\mathcal{S}$ is obtained with the all-solutions mode of a constraint solver and p_j computed by counting. Only enumerations that run to exhaustion are accepted; a time-limited enumeration with solutions already collected would substitute a truncated $\mathcal{S}$ and bias every label. Training states are reachable in-tree states: a depth $d \in \{0, \dots, 12\}$ is drawn uniformly, a prefix of an LP-confidence variable ordering is fixed to the values of a randomly chosen actual solution, and the reduced instance is enumerated in turn to give p_j on its free variables. The network is trained by cross-entropy against the soft targets,

$$\mathcal{L}_1 = -\tfrac{1}{|K|} \sum_{j \in K} [p_j \log \hat{p}_j + (1 - p_j) \log(1 - \hat{p}_j)], \qquad (4)$$

which is minimised exactly at $\hat{p} = p$; replacing p_j by the planted solution, the usual choice, is a noisy surrogate whenever $|\mathcal{S}| > 1$.

4.3.3. Branching rule

At a node the network is evaluated once and the Stage-1 rule branches on the most confident marginal with its majority value,

$$j^\star = \arg\max_{j \in K} |\hat{p}_j - \tfrac{1}{2}|, \qquad v = \mathbb{1}[\hat{p}_{j^\star} \geq \tfrac{1}{2}]. \qquad (5)$$

4.4. Stage 2: exact subtree node counts as the objective

Rule (5) tries first the value that the larger share of surviving solutions takes, so that the first child keeps as many solutions as the marginal can promise. It says nothing about how much the search pays when it does not, nor how much smaller the children become; both are properties of the subtree, and a depth-first search produces them exactly. For a node u let $c(u)$ be the number of nodes expanded below u until a solution is found there or the subtree is exhausted; the search returns it when it backtracks out of u. We use $c(u)$ as the cost of the branching decision made at u; $c(\text{root})$ is the number of nodes the whole search expands to reach a solution, which is the tree-size metric reported throughout the experiments. Stage 2 fine-tunes the branching order on $-\log c(u)$.

4.4.1. Decision process

A state is the reduced instance after propagation; an action is a free variable j; the first value stays $\mathbb{1}[\hat{p}_j \geq \frac{1}{2}]$ from the frozen Stage-1 network. Because each decision owns its subtree, credit is assigned per decision rather than per episode, the property a sparse solved/unsolved reward lacks and the one that makes the problem well posed (Etheve et al., 2020).

4.4.2. Policy and value heads

On the frozen trunk of f_θ two heads are added (Figure 4). The policy logit is

$$g_j(s) = \log\left|\hat{p}_j - \tfrac{1}{2}\right| + h_\phi(\mathbf{h}_j), \qquad (6)$$

with h_ϕ a two-layer network whose last layer is initialised to zero, so that $\pi(j \mid s) \propto \exp(g_j/\tau)$ starts as a softening of rule (5) and its argmax is exactly that rule. A value head $V_\psi(s)$ on the mean- and max-pooled variable and constraint embeddings, $\log|K|$ and $\log m$ predicts $\log c$.

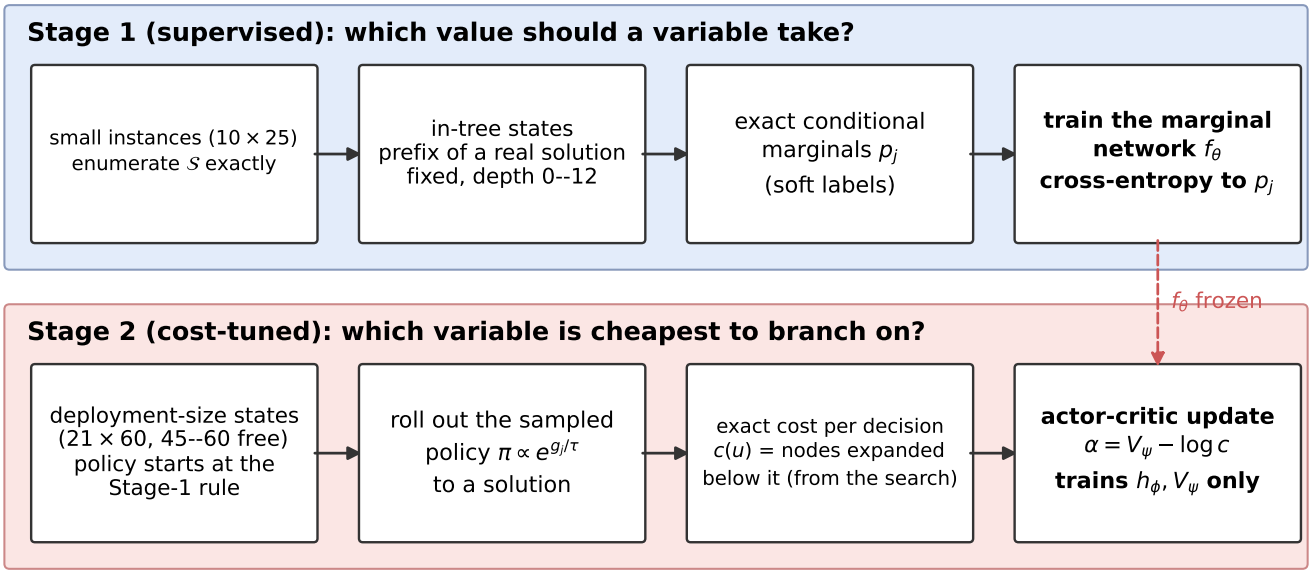


Figure 5: The two training stages. Stage 1 learns *which value* from exact conditional marginals of enumerable instances; Stage 2 learns *which variable* from the exact number of nodes every decision costs, produced by the search, with the Stage-1 network frozen.

4.4.3. Update

Each epoch samples 400 training states with 45–60 free variables, rolls the sampled policy ($\tau = 1$) out to a solution, and records every decision with its exact cost, about 21,000 decisions per epoch. With the advantage $\alpha(u) = V_\psi(s_u) - \log c(u)$ standardised within the epoch,

$$\mathcal{L}_2 = \sum_u \Big[-\alpha(u) \log \pi(j_u \mid s_u) - \lambda \mathcal{H}(\pi(\cdot \mid s_u)) + (V_\psi(s_u) - \log c(u))^2 \Big], \qquad \lambda = 0.01, \tag{7}$$

an on-policy actor-critic update with Monte-Carlo returns (Williams, 1992); no bootstrapping is needed because the return is exact. Only ϕ, ψ are trained; θ is fixed, so the marginal that chooses values and defines the FORCED/OPEN partition is unchanged, and any improvement is attributable to the branching order alone. The critic is pre-trained on decisions recorded under rule (5), and its held-out rank correlation with $\log c$ gates the actor update, since a baseline that cannot rank costs turns the advantage into noise. The epoch is selected by the total node count below held-out states of the training pool, never on evaluation instances. At deployment the policy is greedy, $j^\star = \arg\max_j g_j(s)$.

4.5. Cost per node

At a node with $|K|$ free variables the guided search pays one warm-started relaxation and one forward pass. The forward pass runs on a dense adjacency tensor through TorchScript (the graphs have at most 80 variables and 28 rows, so sparse message passing buys nothing), single-threaded, in 0.3–0.5 ms depending on the number of free variables; the Stage-2 heads add nothing measurable. A single-relaxation classical rule pays the relaxation alone, so the learned rule's node is 1.3–1.5× as expensive, with the ratio falling as the relaxation grows with size (Section 5.2, Table 4). The sound procedure that detects the same FORCED variables, LP probing (fix x_j to each value and prune the one that makes the relaxation infeasible), pays $|K|$ relaxations per node.

5. Experiments

The experiments answer the four questions of Section 1 in turn (Sections 5.2–5.5), after the setup.

5.1. Setup

5.1.1. Instances

Instances are generated as in Section 3.1. Sizes are 10×25, 18×50, 21×60, 24×70 and 28×80 with constraint density $m/n \approx 0.35$ throughout; the evaluation sets hold 30 instances at the two smallest sizes and 100 at each of the three largest, all disjoint from the training pools by generator seed. Stage 1 trains on in-tree states of 1,600 separate 10×25 instances, Stage 2 on states of 500 separate 21×60 instances. At $n = 100$ no method, learned or not, solves an instance within 20 s of CP-SAT, so $n = 80$ is the last size.

5.1.2. Training

Stage 1: 1,600 instances of 10×25 enumerated to exhaustion, eight in-tree states per instance at depths 0–12, 10,000 training and 2,000 test states; Adam, learning rate 10^{-3}, 20 epochs, batch 1, gradient clipping 1.0, about 20 minutes on one CPU core, no hyperparameter tuned and no validation split. Its accuracy on FORCED variables of the test states is 79.1% against a

Bayes ceiling of 83.3% and 76.0–77.5% for relaxation rounding. Stage 2: critic pre-training on decisions recorded under the Stage-1 rule on 21×60 states with 45–60 free variables (held-out Spearman correlation with $\log c$ of 0.81, against 0.75 for a baseline that knows only the state's size), then six epochs of the update (7); the epoch is selected on held-out training states (epoch 6, at 0.36 of the Stage-1 rule's node count below those states).

5.1.3. Rules

Table 2 lists the rules. The two LP rules are the most-integral and most-fractional vertex rules; maxSD is counting-based search, whose solution density on a 0/1 row is exactly b'_i/r_i and therefore costs nothing to compute; dom/wdeg and impact are the two standard adaptive rules of constraint programming, implemented with the failure and reduction statistics of our own loop; LP probing is what strong branching reduces to without an objective; the uniform rule checks that the instances are hard. The oracle enumerates $\mathcal{S}(F, \mathbf{v})$ at every node and is run where that is affordable; the BP rule runs sum-product with exact cardinality-factor messages, damping 0.9 and up to 300 iterations. The learned baselines of Section 2 appear as follows: LP probing stands for imitation of strong branching (it is the expert itself), BP for message-passing marginals, and in Section 5.4 cost-tuned policies started from a random head and from the LP rule stand for reinforcement learning from scratch and from a classical rule.

5.1.4. Protocol and statistics

Two metrics are recorded per instance: the number of nodes expanded to reach a solution, c(root) of Section 4.4, which measures branching quality and is seed-invariant because the search is deterministic given the relaxation vertex; and single-threaded wall-clock time, which also reflects the cost per node. Budgets are 60, 300, 600, 1,200 and 1,200 s per instance for the five sizes, enforced by process termination; six instances run concurrently on a 20-core Intel Core Ultra 7 265KF machine with 32 GB of memory, no GPU is used, and the learned arms use the dense TorchScript path of Section 4.5. The loop is Python 3.10 with NumPy 2.2; the relaxation is solved by Gurobi 13.0 as an LP engine; the networks are PyTorch 2.10; the solver comparisons use SCIP 10.0 through PySCIPOpt 6.1, OR-Tools CP-SAT 9.15, Gurobi 13.0 and fpylll 0.6. Comparisons are paired by instance: we report the median of the per-instance ratios on the instances both rules solved, a 95% bootstrap interval of that median (1,000 resamples of instances), the number of instances on which each rule wins, and a two-sided exact sign test with ties excluded; PAR-2 (mean time with an unsolved instance counted at twice the budget) summarises solve rate and time together. Medians of raw node and time values are given for orientation and are not what the tests act on. No correction for multiple comparisons is applied; with about sixty paired tests in the paper, individual p-values near 0.05 should be read as suggestive and the conclusions rest on the comparisons with $p < 10^{-3}$. The timing of the learned arms is replicated over two further evaluation seeds in Appendix Appendix D.

5.2. RQ1: against classical branching and transplanted learned branching

5.2.1. Tree size

Table 3 and Figure 6 give the main result. At each of the three sizes the learned rule expands fewer nodes than every single-relaxation classical rule, and the paired test is significant for every one of them. Against the strongest cheap rule at each size (dom/wdeg at 21×60 and 24×70, maxSD at 28×80) the median per-instance ratio is $1.52\times$ ($p = 0.035$), $1.79\times$ ($p = 0.035$) and $3.20\times$ ($p = 10^{-9}$). Against most-fractional branching, the rule that a MIP solver uses when it has no pseudocosts, it is $2.27\times$, $3.67\times$ and $3.98\times$ ($p \le 2 \times 10^{-4}$). Against the most-integral LP rule it is $24\times$ and $38\times$ at the two smaller sizes; at 28×80 that rule solves 40/100 instances within budget and expands $19\times$ more nodes on those (PAR-2 1,688 against 75 s). The gains are distributional, not universal: the learned rule wins on 61–80 of 100 instances against the cheap rules and loses on the rest. The one rule that builds a smaller tree is LP probing, by 2.7–2.9× on the instances it solves. It is the transplant of strong branching: at every node it fixes every variable the relaxation can refute, which is more information per node than any score, and it pays $|K|$ relaxations per node for it. Stage 1 alone, the marginal rule without the cost-tuned order, expands $2.3\times$, $3.2\times$ and $5.0\times$ more nodes than the full rule at the three sizes (paired medians). It is itself about $12\times$ below the LP rule at both 21×60 and 24×70, level with the cheap classical rules at 21×60, behind them from 24×70 on, and it leaves 10 of the 28×80 instances unsolved. The marginal alone is thus a competitive value signal and an ordinary variable score; the cost-tuned order is what moves the rule ahead of the field.

5.2.2. Wall-clock

A smaller tree buys time only if the per-node cost allows, and Table 4 measures that cost by instrumenting whole trees. A single-relaxation rule's node costs 0.47, 0.55 and 0.61 ms at the three sizes, 90% of it in the warm-started relaxation. The learned rule's node costs 0.71, 0.73 and 0.82 ms, of which the forward pass is 28–32% and the relaxation 57–62%. The overhead ratio is therefore $1.5\times$ at $n = 60$ and $1.3\times$ at $n = 70$ and 80, because the forward pass grows more slowly with size than the relaxation, while the node advantage rises from $1.5\times$ to $3.2\times$; wall-clock follows (Table 3). At 21×60 the learned rule is as fast as most-fractional and maxSD branching and slower than dom/wdeg ($0.83\times$, $p = 0.021$). At 24×70 it is faster than most-fractional ($2.4\times$, $p = 4 \times 10^{-4}$) and maxSD ($1.5\times$, $p = 0.004$) and level with dom/wdeg. At 28×80 it is faster than all of them, by $3.1\times$, $2.4\times$ and $2.0\times$ ($p \le 0.002$), with 100/100 solved against 99 for most-fractional and 91 for uniform branching, and it has the lowest PAR-2 of the field. LP probing pays 13–18 ms per node, one relaxation per free variable, and is 5–8× slower than the learned rule at the three sizes despite its smaller tree; at 28×80 it leaves 22 instances unsolved (PAR-2 820 against 75 s). Impact-based search, which also probes, is $7\times$ slower than the learned rule at 21×60 and $19\times$ at 24×70 while expanding 6–9× the nodes (paired medians); at 28×80 it solves 49/100 (PAR-2 1,428 s). At the two small sizes (10×25,

Table 2: Branching rules compared. All run in the same depth-first loop (Algorithm 1: one warm-started relaxation per node, the same propagation, the same budget) and differ only in step 5. The single-relaxation classical rules cost one relaxation per node; LP probing costs one per free variable; the reference signals are not deployable and bound what a value signal can do.

group	rule	branches on	first value
classical	LP rule (most-integral)	$\arg\max_j \lvert x_j^{LP} - \frac{1}{2}\rvert$	round(x_j^{LP})
	most-fractional	$\arg\min_j \lvert x_j^{LP} - \frac{1}{2}\rvert$	round(x_j^{LP})
	maxSD (Pesant et al., 2012)	max. solution density of a row	the dense value
	dom/wdeg (Boussemart et al., 2004)	max. weighted failure degree	round(x_j^{LP})
	impact (Refalo, 2004)	max. averaged search-space reduction	the smaller-impact value
	LP probing (= strong branching on a BLS)	fixes refutable values, then LP rule	round(x_j^{LP})
	uniform	a uniformly random free variable	uniform
reference	oracle marginals	$\arg\max_j \lvert p_j - \frac{1}{2}\rvert$, p_j exact by enumeration	$\mathbb{1}[p_j \geq \frac{1}{2}]$
	BP marginals (NSNet-based)	as above with damped sum-product $\tilde{p}_j$	$\mathbb{1}[\tilde{p}_j \geq \frac{1}{2}]$
learned	Stage 1	most confident marginal, $\arg\max_j \lvert \hat{p}_j - \frac{1}{2}\rvert$	$\mathbb{1}[\hat{p}_j \geq \frac{1}{2}]$
	Stage 1+2	cost-tuned score, $\arg\max_j g_j$	$\mathbb{1}[\hat{p}_j \geq \frac{1}{2}]$

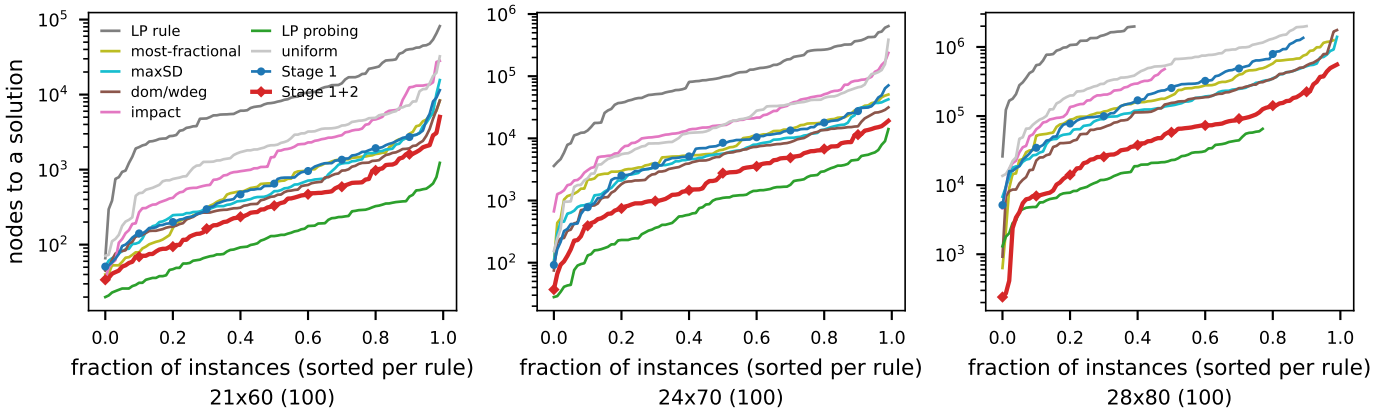


Figure 6: Nodes to a solution on the three 100-instance sets, each rule's instances sorted separately; a curve below another is a smaller tree at every quantile. Only LP probing, at one relaxation per free variable per node, lies below the learned rule.

18×50; 30 instances, trees of 14–120 nodes) every rule is within a few hundredths of a second. The learned rule is the slowest at 10×25, where a 14-node tree cannot amortise 14 forward passes. Those sizes are reported in Appendix Appendix A; they are not where the question is decided.

5.2.3. *Reference signals*

Two rules in Table 2 are not deployable and bound what a value signal can do. The oracle enumerates $\mathcal{S}(F, \mathbf{v})$ at every node and branches on the most confident exact marginal. At 10×25 it expands 22 nodes, against 21 for the Stage-1 network, 14 for the cost-tuned rule and 15.5 for dom/wdeg. The exact value signal under the confidence rule is therefore no better than its learned approximation and worse than a classical variable score, so the remaining headroom is in the variable choice, which is what Stage 2 addresses. Belief-propagation marginals, the NSNet transplant, give trees the size of Stage 1's at 18×50 (118 against 116 nodes), but one estimate costs 35 ms against 0.3–0.5 ms for a dense forward pass, which makes the rule 50× slower than the learned one in wall-clock. The marginal network is this signal at a fraction of its price, and it is run once per node, which message passing at this cost cannot be. NSNet's own way of using a marginal, decoding one root estimate into an initial assignment for an incomplete local search, was also run. A WalkSAT-style search over $\{0,1\}^n$ on the residual $\|A\mathbf{x} - \mathbf{b}\|_1$ solves all 100 instances at 21×60 in 1.2–1.5 s, 3–4× the time of the learned search. It takes the same time whether its initial assignment comes from the marginal network, from LP rounding or from a uniform draw. At this size the marginal buys the local search nothing, and the comparison is between the two searches, not between the two uses of the signal.

5.3. *RQ2: generalisation to larger instances*

Both stages were trained below the sizes that matter: the marginal network on 10×25 states and the branching order on 21×60 states. The architecture is size-free, a graph network over the reduced instance, and the training signals are defined at any size. Training a search heuristic on small instances and deploying it on larger ones is the usual regime for learned search in operations research (Hottung et al., 2020; Yan et al., 2025), and the question is whether the advantage survives the extrapolation. Figure 7 and Table 3 answer it in two parts. First, the

Table 3: Branching rules in the same search loop, 100 instances per size. Medians are over solved instances; the last two columns are the median per-instance ratio of the rule to Stage 1+2 on commonly solved instances, its 95% bootstrap interval, and the two-sided sign test p-value (ties excluded). A ratio above 1 means the rule expands more nodes or takes longer than the learned rule. PAR-2 counts an unsolved instance at twice the budget.

rule	solved	med. nodes	med. s	PAR-2 s	nodes, rule / Stage 1+2	time, rule / Stage 1+2
21×60 (100), budget 600 s						
LP rule (most-integral)	100/100	7,762	4.58	8.0	23.90× [15.11, 37.29] 10^{-25}	10.55× [7.68, 18.48] 10^{-22}
most-fractional	100/100	662	0.41	0.7	2.27× [1.29, 4.39] 10^{-4}	1.13× [0.79, 1.57] 0.48
maxSD	100/100	494	0.30	0.7	1.52× [1.13, 2.45] 10^{-3}	0.77× [0.61, 1.32] 0.48
dom/wdeg	100/100	436	0.25	0.5	1.52× [1.08, 1.95] 0.04	0.83× [0.61, 0.96] 0.02
impact	100/100	1,464	2.68	5.8	5.76× [2.69, 7.72] $4 \cdot 10^{-9}$	7.06× [3.98, 12.84] 10^{-15}
LP probing	100/100	130	1.99	2.9	0.37× [0.28, 0.44] 10^{-6}	5.36× [4.47, 6.32] 10^{-22}
uniform	100/100	2,112	1.21	2.0	5.12× [3.66, 9.31] 10^{-14}	2.81× [1.98, 4.26] 10^{-10}
Stage 1	100/100	628	0.65	1.2	2.32× [1.08, 3.25] 0.01	2.02× [1.19, 2.83] 10^{-3}
Stage 1+2	100/100	**325**	**0.36**	0.6	—	—
24×70 (100), budget 1200 s						
LP rule (most-integral)	100/100	95,440	51.95	81.5	37.74× [27.17, 66.01] $8 \cdot 10^{-27}$	21.88× [16.38, 40.52] $8 \cdot 10^{-27}$
most-fractional	100/100	6,504	3.97	6.8	3.67× [2.13, 4.69] $6 \cdot 10^{-11}$	2.43× [1.42, 2.94] $4 \cdot 10^{-4}$
maxSD	100/100	5,637	3.23	5.4	2.46× [1.71, 3.10] 10^{-7}	1.46× [1.10, 1.87] $4 \cdot 10^{-3}$
dom/wdeg	100/100	5,263	2.73	3.9	1.79× [1.15, 2.42] 0.04	1.09× [0.68, 1.37] 0.48
impact	100/100	16,256	33.07	64.7	8.92× [5.57, 10.95] $8 \cdot 10^{-14}$	19.01× [12.89, 25.36] $6 \cdot 10^{-24}$
LP probing	100/100	1,088	14.27	23.4	0.36× [0.24, 0.57] 10^{-3}	5.17× [3.40, 7.94] $6 \cdot 10^{-11}$
uniform	100/100	16,028	9.70	18.0	6.76× [4.55, 9.10] 10^{-14}	4.15× [3.02, 5.50] $6 \cdot 10^{-11}$
Stage 1	100/100	8,174	6.60	9.1	3.17× [2.32, 4.40] 10^{-6}	2.63× [2.10, 3.85] $5 \cdot 10^{-6}$
Stage 1+2	100/100	**2,580**	**2.39**	3.6	—	—
28×80 (100), budget 1200 s						
LP rule (most-integral)	40/100	1,029,039	622.27	1688.2	19.25× [13.83, 27.87] 10^{-9}	14.60× [10.28, 22.08] 10^{-9}
most-fractional	99/100	187,381	114.49	221.3	3.98× [2.41, 5.74] $7 \cdot 10^{-9}$	3.07× [1.96, 4.63] $9 \cdot 10^{-7}$
maxSD	100/100	141,178	88.12	135.8	3.20× [2.29, 4.79] 10^{-9}	2.39× [1.70, 3.51] $5 \cdot 10^{-6}$
dom/wdeg	100/100	152,598	81.46	119.3	3.15× [2.47, 4.17] 10^{-7}	1.96× [1.57, 2.65] 10^{-3}
impact	49/100	149,624	357.32	1428.4	2.50× [1.69, 3.50] 10^{-3}	7.21× [4.97, 10.09] $6 \cdot 10^{-8}$
LP probing	78/100	17,148	309.62	819.8	0.35× [0.21, 0.47] $6 \cdot 10^{-7}$	7.71× [4.65, 10.17] 10^{-15}
uniform	91/100	520,855	289.99	537.4	7.28× [5.93, 9.77] $7 \cdot 10^{-18}$	5.02× [4.08, 6.86] 10^{-13}
Stage 1	90/100	212,271	182.65	485.0	5.01× [3.01, 6.62] $9 \cdot 10^{-11}$	5.30× [3.22, 6.89] 10^{-11}
Stage 1+2	100/100	**58,026**	**46.61**	74.5	—	—

Table 4: Per-node cost on the dense inference path: six instances per size, the whole tree of each rule instrumented (median over instances of milliseconds per node and of the share of node time spent in the warm-started relaxation and in the network forward pass; the remainder is reduction, propagation and bookkeeping). The single-relaxation row is the LP rule (most-integral branching); every single-relaxation classical rule in Table 2 shares its per-node cost because the score is free once the relaxation is solved. One forward pass costs 0.33–0.53 ms and grows with the number of free variables.

size	rule	ms / node	LP share	forward share
21×60	single-relaxation rule (LP rule)	0.47	90%	0%
21×60	Stage 1	0.67	64%	27%
21×60	**Stage 1+2**	**0.70**	57%	32%
24×70	single-relaxation rule (LP rule)	0.55	91%	0%
24×70	Stage 1	0.75	69%	23%
24×70	**Stage 1+2**	**0.73**	62%	28%
28×80	single-relaxation rule (LP rule)	0.61	92%	0%
28×80	Stage 1	0.86	69%	23%
28×80	**Stage 1+2**	**0.82**	62%	29%

rule transfers: at 24×70 and 28×80, sizes neither stage saw, it solves every instance within budget and keeps a significant node advantage over every single-relaxation rule. Second, the advantage grows with size rather than decaying. The node ratio against the strongest cheap rule rises from 1.5× at $n = 60$ to 1.8× at 70 and 3.2× at 80, and against most-fractional branching from 2.3× to 3.7× and 4.0×. The wall-clock ratio against dom/wdeg crosses one between $n = 60$ (0.83×) and 70 (1.09×) and reaches 2.0× at 80. The mechanism is the one of Table 4: the per-node premium of the forward pass is fixed while the relaxation and the tree grow, so a constant-factor node advantage becomes a time advantage once the tree is large enough. The node advantage itself is larger where the search is longer, which Section 5.4 traces to how the cost-tuned order refutes wrong values. The transfer is uneven between the stages. Stage 1 alone keeps its node advantage over the LP rule at every size, but it falls behind the cheap classical rules in time from 24×70 and in solve rate at 28×80 (90/100). What transfers usefully is the combination, with Stage 2's gain over Stage 1 growing from 2.3× at $n = 60$ to 5.0× at $n = 80$. What transfer does not do is extend the useful window: at $n = 100$ no rule solves an instance within budget, and the next size would need either a cheaper node or clause learning.

5.4. RQ3: are the two signals interchangeable, and is each stage necessary?

5.4.1. Value and variable are separable, and the variable carries the gain

Table 5 and Figure 8 combine each of three variable-selection scores (LP most-integral, Stage-1 confidence, Stage-2 cost-tuned) with each of two first-value rules (LP rounding, Stage-1 marginal). Read by column, the variable score is decisive. With

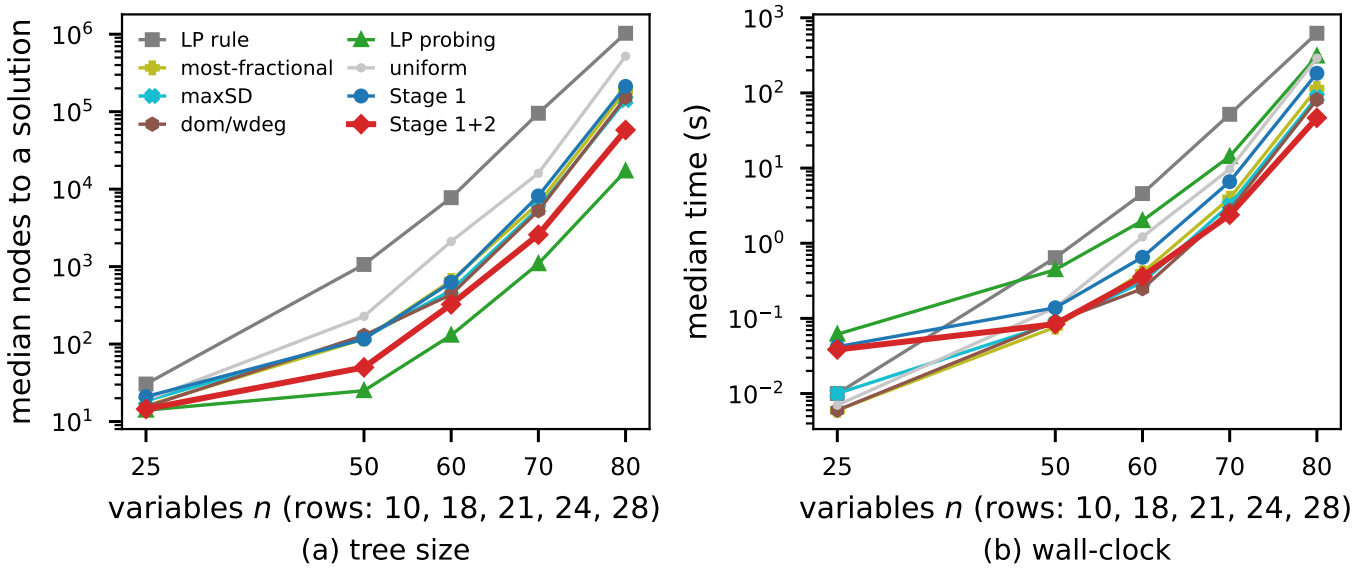


Figure 7: Scaling with size. (a) Median nodes and (b) median wall-clock per rule (30 instances at $n = 25, 50$; 100 at $n = 60, 70, 80$). Stage 1 was trained at $n = 25$ and Stage 2 at $n = 60$, so $n = 70$ and 80 are extrapolation.

Table 5: Value × variable cross ablation: each arm combines one variable-selection score with one first-value rule; everything else is identical. Columns per size: median nodes and the median per-instance node ratio to Stage 1+2 with bootstrap interval and sign-test p-value.

	21×60 (100)		24×70 (100)		28×80 (100)	
variable, value	nodes	ratio	nodes	ratio	nodes	ratio
LP var., LP value (LP rule)	7,762	23.90× [15.11, 37.29] 10^{-25}	95,440	37.74× [27.17, 66.01] $8 \cdot 10^{-27}$	1,029,039	19.25× [13.83, 27.87] 10^{-9}
LP var., S1 value	9,810	27.87× [19.53, 36.44] 10^{-21}	69,872	39.45× [20.99, 58.92] 10^{-21}	647,145	16.32× [6.86, 23.92] $9 \cdot 10^{-5}$
S1 var., S1 value (Stage 1)	628	2.32× [1.08, 3.25] 0.01	8,174	3.17× [2.32, 4.40] 10^{-6}	212,271	5.01× [3.01, 6.62] $9 \cdot 10^{-11}$
S1 var., LP value	956	2.84× [1.77, 3.75] $5 \cdot 10^{-6}$	6,998	3.28× [2.56, 4.22] $6 \cdot 10^{-7}$	278,175	5.85× [4.04, 8.77] $5 \cdot 10^{-12}$
S2 var., S1 value (Stage 1+2)	325	—	2,580	—	58,026	—
S2 var., LP value	292	0.79× [0.60, 1.04] 0.09	2,348	1.19× [0.91, 1.66] 0.27	69,665	1.41× [1.03, 1.86] 0.09

the Stage-1 value fixed, moving the variable score from LP to Stage 1 to Stage 2 divides the tree by 12× and then by 2.3× at 21×60. The LP variable with the exact-trained marginal value is 28× worse than the full rule at 21×60 and 37× worse at 24×70, and it solves 26/100 instances at 28×80 where the full rule solves all. Read by row, the value rule is not decisive. With the Stage-2 variable fixed, replacing the marginal value by LP rounding changes the tree by 0.79× ($p = 0.09$) at 21×60, 1.19× ($p = 0.27$) at 24×70 and 1.41× ($p = 0.09$) at 28×80; none of these is significant at the 5% level, although the direction turns in favour of the marginal value as the tree grows. With the Stage-1 variable fixed, the LP value costs 1.5× and 1.3× more nodes at the two smaller sizes, and that arm falls further behind the full rule with size (2.8×, 3.3×, 5.9×), which is the Stage-2 gain growing with the tree. Two conclusions follow, one of them against the design of Section 4. The signals are not interchangeable in the sense that matters: the marginal-trained network is what makes a good variable score possible, through its trunk and through the confidence score that Stage 2 starts from, and no amount of value accuracy rescues the LP variable choice. But the exact-marginal *value* is replaceable. Once the variable is chosen by the cost-tuned score, rounding the relaxation gives a tree that is not significantly different at any size and at most 1.4× larger at 28×80. This is consistent with the oracle result of Section 5.2, where even exact marginals under the confidence rule do not beat a classical variable score. The value signal's job in this method is to train the representation, not to pick values.

5.4.2. *What Stage 2 learned*

The fine-tuning changes the search in a way that the per-instance records make visible (Table B.10 in Appendix Appendix B). For every rule we record, on the path to the solution, how often the first value tried was refuted (the subtree under it exhausted) and what each such refutation cost. The Stage-1 confidence rule is wrong least often, on 6% of its decisions at 21×60, and pays most per mistake, 246 nodes. The cost-tuned rule is wrong three times as often (19%) and pays a quarter per mistake (65 nodes), and its solution path is shorter (28 against 46 decisions) because the variables it picks trigger more propagation and relaxation fixings. The same holds against the classical rules at 24×70: dom/wdeg, maxSD and most-fractional branching are refuted on 14–16% of their decisions at 835–911 nodes each, the learned rule on 21% at 387. This is fail-first (Haralick and Elliott, 1980) learned from the cost signal rather than designed: the policy moves away from the variables it is

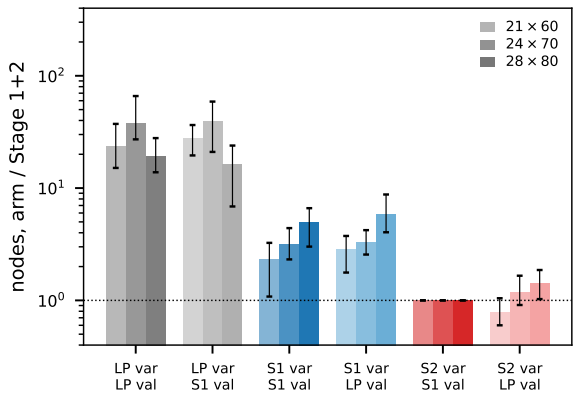


Figure 8: The cross ablation of Table 5 as median per-instance node ratios to Stage 1+2 (log scale, 95% bootstrap intervals), for the three sizes.

surest about towards variables whose wrong value is cheap to exhaust. The arm that keeps the Stage-2 variable with the LP value has the same profile (16%, 63 nodes), so the structure comes from the variable choice.

5.4.3. *Is each stage necessary, and which parts of the recipe matter?*

Table 6 varies the training of each stage with everything else fixed. For Stage 1, nothing in the recipe moves the value accuracy. The label form (soft marginals against the planted 0/1 solution), the training seed and the number of training states (10,000 against 2,771) change the test accuracy on FORCED variables by at most 0.2 points (79.0–79.2% against a ceiling of 83.3%). A network of the NSNet architecture (Li and Si, 2022) trained on the same states reaches 80.4%, and training on 21×60 states instead of 10×25 (M1) raises accuracy on 21×60 states by 2.4 points. The trees tell a different story. Two training seeds of the reference recipe expand 553 and 562 nodes at 21×60 against the reference's 628, so the Stage-1 rule carries a seed variance of about ±10% that no label or architecture choice exceeds in the right direction. The two networks with the *higher* value accuracy, M1 and the NSNet architecture, expand the *largest* trees: 912 and 892 at 21×60, and 10,754 and 12,500 at 24×70 against 6,199–8,175 for the reference recipe. At 28×80 they solve the fewest instances within budget, 85 and 70 of 100, against 91–95 for the other Stage-1 variants and 100 for the full rule, and every Stage-1 variant expands 4.0–5.7× the nodes of the full rule there. Value accuracy and tree size are therefore not the same quantity, which is the point of Table 5 seen from the other side. The Stage-1 recipe is necessary only in the sense that any of these networks supplies a trunk and a confidence score that Stage 2 then improves by 2–3×. For Stage 2 the recipe matters less than we had designed it to. Starting the policy head at random, which is what reinforcement learning from scratch would do with this trunk, leaves the initial policy at 7× the root cost of the marginal rule; six epochs later it is level with the reference (1.04×, 0.86× and 1.03× at the three sizes, none significant). Starting it from the LP rule ends in the same place (1.06×, 0.86×, 1.12×). Unfreezing the trunk makes the held-out node count diverge from the third epoch, to 2.7–5.4× the Stage-1 rule's, but the checkpoint that epoch selection keeps, from the first epoch, is as good as the frozen one (0.76×, 0.83×, 1.06×, not significant). The frozen trunk is thus what keeps the fine-tuning stable enough to continue and what lets the gain be attributed to the order. It is not a source of tree size, and we do not claim one.

5.5. *RQ4: against complete solvers*

5.5.1. *As a branching rule inside SCIP*

The learned rule is a function of the reduced instance and its relaxation vertex, so it can be installed as a branching rule in a MIP solver. We implemented it as a PySCIPOpt branching plug-in and ran SCIP (Bestuzheva et al., 2021) on the same instances with presolve, cutting planes and primal heuristics off, depth-first node selection and a stop at the first solution. SCIP's search is then the one our loop performs, with SCIP's propagation and LP in place of ours. The comparison is against reliability pseudocost branching (RPB, SCIP's default and the rule the learning-to-branch literature imitates, Achterberg et al., 2005), SCIP's most-fractional rule and its inference rule. Conflict analysis is SCIP's clause learning. When a node is found infeasible, by the relaxation or by propagation, the solver extracts from the dual ray or the propagation reasons a minimal set of bound changes that cannot hold together and adds it as a global conflict constraint, so that every later subtree that reaches the same combination is cut without being searched. Our own loop has no such mechanism, so it is run both off and on. Table 7 shows that the node advantage survives the change of engine and the addition of clause learning. On the 100-instance sets the plug-in expands 1.9–3.8× fewer nodes than RPB at every size with conflict analysis off or on (sign test $p \leq 3 \times 10^{-5}$ throughout), and 1.7–4.0× fewer than SCIP's most-fractional and inference rules. Clause learning helps the two rules unequally. With it on, RPB expands 22% fewer nodes at 24×70 and 34% fewer at 28×80, the plug-in only 6% and 3% fewer, so the node ratio against RPB shrinks from 3.5–3.8× to 1.9–3.1× and stays significant. The reading is that a rule which already refutes wrong values shallowly and cheaply produces few long conflicts worth reusing elsewhere, while a bound-driven rule leaves more for the conflict constraints to cut. Time is a draw against RPB and a loss against the cheapest rule. The forward pass is 60–80% of the branching callback, so the plug-in is 3× slower than every SCIP rule at 21×60, where trees are a few hundred nodes. At 24×70 and 28×80 its median time is within 0.8–1.1× of RPB's with conflict analysis off or on (no paired difference significant) and 0.66–0.86× behind SCIP's most-fractional rule ($p = 0.02$–0.06). Inside a solver the answer to RQ1 therefore splits: the learned rule is the better branching rule by tree size at every size, it matches the solver's default rule in time once the tree is large enough to amortise the forward pass, and it does not beat the cheapest rule in time.

Table 6: Training variants: median nodes and median per-instance node ratio to the reference Stage 1+2 rule, per size. Stage-1 variants are run with the confidence rule (no Stage 2); Stage-2 variants share the frozen reference trunk unless stated.

	21×60 (100)		24×70 (100)		28×80 (100)	
variant	nodes	ratio	nodes	ratio	nodes	ratio
Stage 1 (reference)	628	2.32× [1.08, 3.25] 0.01	8,174	3.17× [2.32, 4.40] 10^{-6}	212,271 (90/100)	5.01× [3.01, 6.62] $9 \cdot 10^{-11}$
Stage 1, hard 0/1 labels	848	2.66× [1.93, 4.05] $3 \cdot 10^{-5}$	7,644	2.73× [2.03, 3.98] $4 \cdot 10^{-9}$	259,605 (95/100)	4.18× [3.26, 6.63] 10^{-12}
Stage 1, training seed 1	562	1.72× [1.17, 2.49] $4 \cdot 10^{-3}$	7,318	2.72× [1.60, 4.60] $4 \cdot 10^{-4}$	207,117 (91/100)	4.03× [2.63, 5.13] 10^{-10}
Stage 1, training seed 2	553	1.65× [1.10, 2.14] 0.04	6,199	2.06× [1.01, 2.76] 0.09	213,055 (95/100)	4.37× [3.09, 6.74] $3 \cdot 10^{-11}$
Stage 1, 2,771 states (size ctrl.)	662	2.22× [1.47, 2.61] $8 \cdot 10^{-5}$	6,897	3.89× [1.96, 6.15] $3 \cdot 10^{-5}$	168,442 (91/100)	4.12× [3.08, 6.44] 10^{-9}
Stage 1 on 21×60 states (M1)	912	2.44× [1.64, 4.10] 10^{-5}	10,754	3.94× [2.51, 7.91] 10^{-7}	225,182 (85/100)	5.73× [2.99, 7.19] $8 \cdot 10^{-8}$
Stage 1, NSNet architecture	892	2.75× [2.02, 4.03] 10^{-7}	12,500	5.02× [3.46, 7.46] $5 \cdot 10^{-13}$	248,556 (70/100)	4.75× [3.45, 7.73] 10^{-8}
Stage 1+2 (reference)	325	—	2,580	—	58,026	—
Stage 2 from a random head	333	1.04× [0.74, 1.28] 0.92	1,878	0.86× [0.65, 1.10] 0.27	59,122 (99/100)	1.03× [0.94, 1.20] 0.84
Stage 2 from the LP rule	376	1.06× [0.73, 1.31] 0.76	3,024	0.86× [0.62, 1.39] 0.48	72,539	1.12× [0.87, 1.48] 0.37
Stage 2, trunk unfrozen	280	0.76× [0.56, 1.20] 0.37	2,427	0.83× [0.69, 1.12] 0.37	59,692	1.06× [0.70, 1.65] 0.76

5.5.2. End to end

Table 8 places the learned search next to complete solvers with all their defaults (Gurobi with one thread (Gurobi Optimization, LLC, 2024); CP-SAT with one worker (Perron and Didier, 2023); SCIP with presolve, cuts, heuristics, conflict analysis and RPB) and next to lattice reduction with bounded enumeration (BKZ block 40, 30 column permutations, fpylll enumeration (The FPLLL development team, 2023) with a 300 s cap), on the same instances and budgets. Every method solves every instance. Gurobi is the fastest at every size (0.05, 0.31 and 6.6 s median at $n = 60$, 70, 80) and CP-SAT the next (0.04, 0.36, 12.2 s, with a heavier tail at 28×80). SCIP with its defaults is level with the learned search at 24×70 (2.33 against 2.39 s) and 2.8× faster at 28×80 (16.5 against 46.6 s). The gap between the two MIP solvers, 2.5–8× here, is not a setting of ours. Both run with all defaults on one thread; the difference is in the LP engine and the search implementation, which pay off on instances the search closes quickly (SCIP's own statistics show presolve and primal heuristics finding nothing on these instances). The plug-in comparison of Table 7 runs SCIP with presolve and cuts off and is a different measurement. Lattice enumeration solves the 21×60 set in 0.2 s per instance, but its cost grows faster than the search's: 13.2 s at 24×70 and 125 s at 28×80, 5.5× and 2.7× slower than the learned search. Two things are true at once. The learned search is not the fastest way to solve these instances: a commercial MIP solver and a CP-SAT solver with presolve and clause learning are ahead by one to two orders of magnitude at $n \leq 70$ and by 4–7× at $n = 80$, and SCIP is ahead at the largest size. And none of that bears on the branching question. The comparison that isolates the branching decision is Table 3, and inside SCIP the decision quality carries over while the per-node cost does not. The honest summary is that this paper improves the branching rule, in its own loop and inside SCIP, and does not by itself produce a faster solver. The gap to the solvers is clause learning and a compiled node, neither of which the learned rule addresses.

6. Conclusion

A branching decision asks two questions, and this paper trained on an exact answer to each: the conditional marginal, enumerable where solutions can be enumerated and carried to every node by the reduction identity, for which value to try; and the number of nodes below a decision, produced by the search as it backtracks, for which variable to branch on. Against seven classical rules and the transplants of three learned-branching families, in one search loop, on 100-instance sets, the learned rule expands the fewest nodes of any single-relaxation rule at every size and is also the fastest at 28×80; trained at $n \leq 60$, its advantage grows with size rather than decaying; and it carries its node advantage into SCIP against reliability branching with conflict analysis on or off. The ablations say what the recipe is: the variable choice carries the gain, the cost-tuned order is fail-first learned rather than designed, the frozen marginal trunk is what keeps the fine-tuning stable, and the exact-marginal value turns out to be replaceable by LP rounding once the variable is chosen well, with at most a 1.4× difference at the largest size.

Three limitations bound the result. The claims are for one instance family, the planted 0/1 market-split construction of Section 3.1, and whether they hold for other constraint families or for projection matrices with the structure of a real scanner is untested. The learned search is not competitive end to end: Gurobi, CP-SAT and SCIP with their defaults are faster, by orders of magnitude at $n \leq 70$, so the contribution is to the branching rule and not to the solver; lattice enumeration also solves every instance we study, though at the two larger sizes it is slower than the learned search. And exact supervision bounds Stage 1 to sizes that enumerate, so the value signal cannot be trained where it is deployed; that it does not need to be, because the LP value serves, is the one result that eases this.

The next steps follow from the negative results. If the value signal's job is to train the representation, a cheaper target than exact marginals may do, and Stage 2's cost signal needs no enumeration at all, so the order can be fine-tuned at sizes where enumeration fails and, since the number of nodes below a decision is defined whether or not the subtree holds a solution, on infeasible instances, which this paper excludes. The time gap inside a solver is a forward-pass budget: the plug-in loses on time with the network at 60–80% of the callback, so either a smaller network or a learned rule evaluated at a subset of nodes is the route to a rule that wins on both counts.

Table 7: Inside SCIP (presolve, cuts and primal heuristics off, depth-first node selection, first solution stops): SCIP's own rules against the learned rule as a branching plug-in, with conflict analysis off and on. Ratios are rule / plug-in as in Table 3.

conflict	rule	solved	med. nodes	med. s	nodes, rule / plug-in	time, rule / plug-in
21×60 *(100)*						
off	RPB (SCIP default)	100/100	828	0.06	2.14× [1.57, 3.42] $5 \cdot 10^{-6}$	0.34× [0.24, 0.51] $6 \cdot 10^{-7}$
off	most-fractional (SCIP)	100/100	558	0.05	1.83× [1.37, 2.39] 10^{-4}	0.26× [0.20, 0.39] 10^{-15}
off	inference (SCIP)	100/100	767	0.06	2.41× [1.77, 3.80] $8 \cdot 10^{-5}$	0.32× [0.23, 0.45] $6 \cdot 10^{-11}$
off	**Stage 1+2 plug-in**	100/100	**274**	**0.14**	—	—
21×60 *(100)*						
on	RPB (SCIP default)	100/100	705	0.06	1.86× [1.46, 2.81] $3 \cdot 10^{-5}$	0.32× [0.25, 0.46] 10^{-9}
on	most-fractional (SCIP)	100/100	528	0.04	1.69× [1.33, 2.27] 10^{-4}	0.28× [0.23, 0.39] 10^{-11}
on	inference (SCIP)	100/100	795	0.07	2.09× [1.62, 2.84] 10^{-5}	0.33× [0.24, 0.45] 10^{-12}
on	**Stage 1+2 plug-in**	100/100	**256**	**0.14**	—	—
24×70 *(100)*						
off	RPB (SCIP default)	100/100	7,010	0.56	3.47× [2.42, 6.00] $6 \cdot 10^{-11}$	0.83× [0.58, 1.37] 0.37
off	most-fractional (SCIP)	100/100	5,026	0.40	3.47× [1.59, 4.23] $3 \cdot 10^{-5}$	0.86× [0.41, 0.98] 0.04
off	inference (SCIP)	100/100	6,694	0.50	4.00× [2.48, 6.36] 10^{-9}	0.95× [0.60, 1.37] 0.48
off	**Stage 1+2 plug-in**	100/100	**1,915**	**0.61**	—	—
24×70 *(100)*						
on	RPB (SCIP default)	100/100	5,496	0.49	3.06× [2.57, 4.53] $6 \cdot 10^{-7}$	0.80× [0.64, 1.05] 0.13
on	most-fractional (SCIP)	100/100	5,340	0.50	2.98× [1.67, 3.87] 10^{-5}	0.74× [0.48, 1.03] 0.13
on	inference (SCIP)	100/100	5,578	0.48	3.12× [2.26, 4.34] 10^{-9}	0.78× [0.58, 0.97] 0.02
on	**Stage 1+2 plug-in**	100/100	**1,806**	**0.60**	—	—
28×80 *(100)*						
off	RPB (SCIP default)	100/100	224,088	25.05	3.83× [2.36, 5.96] 10^{-10}	1.11× [0.71, 1.73] 0.48
off	most-fractional (SCIP)	100/100	136,840	15.84	2.11× [1.63, 3.01] $7 \cdot 10^{-3}$	0.66× [0.49, 0.93] 0.02
off	inference (SCIP)	100/100	232,221	25.19	3.90× [3.00, 5.70] 10^{-10}	1.14× [0.84, 1.59] 0.62
off	**Stage 1+2 plug-in**	100/100	**60,174**	**22.42**	—	—
28×80 *(100)*						
on	RPB (SCIP default)	100/100	148,872	19.75	2.43× [1.80, 4.76] $3 \cdot 10^{-5}$	0.77× [0.57, 1.43] 0.37
on	most-fractional (SCIP)	100/100	128,310	17.10	2.08× [1.30, 2.76] 10^{-3}	0.68× [0.41, 0.96] 0.06
on	inference (SCIP)	100/100	122,144	14.62	2.36× [1.52, 3.00] 10^{-4}	0.71× [0.46, 0.90] 0.01
on	**Stage 1+2 plug-in**	100/100	**58,564**	**24.00**	—	—

Appendix A. The two small sizes

Table A.9 reports 10×25 and 18×50. The reference signals appear here because the oracle enumerates $\mathcal{S}(F, \mathbf{v})$ at every node and is affordable only at 10×25, and belief propagation finishes within budget at 18×50.

Appendix B. Refutation structure of every rule

Table B.10 records, for every rule and every solved instance, how the search spent its nodes. The *path* is the number of branching decisions on the way to the solution; the *refuted share* is the fraction of all expanded nodes that lie in subtrees exhausted without a solution; the *first-child failure* is the fraction of path decisions whose first value was refuted, that is, how often the value choice was wrong; the *refutation cost* is the mean number of nodes such a wrong first child cost before it was exhausted; and the *depth fraction* is the median depth of those wrong decisions as a fraction of the path length. Three readings carry the argument of Section 5.4. First, every rule spends more than 90% of its nodes in refuted subtrees at 21×60 and more than 99% at 28×80, so a rule is judged by how cheaply it refutes, not by how rarely it is wrong. Second, the Stage-1 rule is the rule that is wrong least often and pays most per mistake, while the cost-tuned rule is wrong 2–3× more often and pays 3–5× less; the arms that keep the Stage-2 variable with the LP value have the same profile, so the change comes from the variable choice. Third, the cost-tuned rule's solution path is the shortest of the single-relaxation rules at every size, because the variables it picks trigger more propagation and relaxation fixings; LP probing has a shorter path still, because it fixes variables directly.

Appendix C. Implementation of the classical rules

All rules share Algorithm 1 and differ only in step 5; each receives the reduced instance $(A', \mathbf{b}')$ and the relaxation vertex $\mathbf{x}^{LP}$. *Most-integral* and *most-fractional* branch on $\arg\max_j |x_j^{LP} - \frac{1}{2}|$ and $\arg\min_j |x_j^{LP} - \frac{1}{2}|$ with the rounded value first. *maxSD*: on a 0/1 equality row with r_i free variables and right-hand side b_i' the solution density of $x_j = 1$ is b_i'/r_i for every variable of the row and that of $x_j = 0$ is $1 - b_i'/r_i$; the rule takes, over all rows and variables, the variable–value pair of maximum density, ties broken by the sum of densities over the rows containing the variable. *dom/wdeg*: every free variable has domain size 2, so the rule reduces to the maximum conflict-weighted degree; each original row carries a weight, initially 0, incremented whenever the bound check or propa-

Table 8: End to end, same instances and budgets, single-threaded: complete solvers with all defaults, lattice reduction with bounded enumeration, and the learned search. Median time over solved instances and PAR-2; node counts are not comparable across engines and are omitted.

method	solved	med. s	PAR-2 s
21×60 *(100)*			
SCIP (all defaults)	100/100	0.35	0.5
CP-SAT (1 worker)	100/100	0.04	0.1
Gurobi (1 thread)	100/100	0.04	0.1
BKZ-40 × 30 + enumeration	100/100	0.20	0.4
Stage 1+2 (our loop)	100/100	**0.36**	0.6
24×70 *(100)*			
SCIP (all defaults)	100/100	2.33	2.5
CP-SAT (1 worker)	100/100	0.36	0.6
Gurobi (1 thread)	100/100	0.31	0.4
BKZ-40 × 30 + enumeration	100/100	13.20	11.5
Stage 1+2 (our loop)	100/100	**2.39**	3.6
28×80 *(100)*			
SCIP (all defaults)	100/100	16.47	21.1
CP-SAT (1 worker)	100/100	12.24	39.8
Gurobi (1 thread)	100/100	6.61	9.8
BKZ-40 × 30 + enumeration	100/100	124.90	117.7
Stage 1+2 (our loop)	100/100	**46.61**	74.5

Table A.9: The two small sizes (30 instances each): same columns as Table 3. Trees are tens of nodes and no rule separates from the others in time except through the cost of its score.

rule	solved	med. nodes	med. s	PAR-2 s	nodes, rule / Stage 1+2	time, rule / Stage 1+2
10×25 (30), budget 60 s						
LP rule (most-integral)	30/30	30	0.01	0.0	1.89× [1.53, 2.23] $7 \cdot 10^{-9}$	0.23× [0.20, 0.28] 10^{-9}
most-fractional	30/30	16	0.01	0.0	1.03× [0.83, 1.18] 0.70	0.16× [0.13, 0.17] 10^{-9}
maxSD	30/30	18	0.01	0.0	1.24× [1.18, 1.46] 10^{-5}	0.27× [0.20, 0.30] 10^{-9}
dom/wdeg	30/30	16	0.01	0.0	1.03× [0.93, 1.19] 0.85	0.16× [0.12, 0.18] 10^{-9}
impact	30/30	17	0.03	0.0	1.14× [1.07, 1.28] 0.02	0.75× [0.67, 0.89] $6 \cdot 10^{-5}$
LP probing	30/30	14	0.06	0.1	0.94× [0.81, 1.07] 0.34	1.43× [1.28, 1.63] 10^{-4}
uniform	30/30	19	0.01	0.0	1.23× [1.10, 1.41] 10^{-4}	0.19× [0.15, 0.21] 10^{-9}
oracle marginals	30/30	22	0.19	0.2	1.47× [1.39, 1.63] 10^{-9}	4.52× [4.14, 5.41] 10^{-9}
BP marginals (NSNet-based)	30/30	21	0.41	0.4	1.47× [1.32, 1.63] $4 \cdot 10^{-9}$	10.06× [9.01, 11.44] 10^{-9}
Stage 1	30/30	21	0.04	0.0	1.45× [1.30, 1.55] $4 \cdot 10^{-9}$	1.10× [1.05, 1.14] 0.02
Stage 1+2	30/30	**14**	**0.04**	0.0	—	—
18×50 (30), budget 300 s						
LP rule (most-integral)	30/30	1,066	0.64	0.9	13.89× [7.69, 42.45] 10^{-9}	4.71× [3.68, 12.60] $9 \cdot 10^{-7}$
most-fractional	30/30	116	0.08	0.1	2.23× [1.04, 3.83] 0.10	0.82× [0.62, 1.12] 0.36
maxSD	30/30	120	0.09	0.1	2.18× [1.07, 3.26] 0.04	0.93× [0.74, 1.33] 0.58
dom/wdeg	30/30	130	0.09	0.1	2.39× [1.27, 4.62] 0.02	0.90× [0.52, 1.89] 1.00
impact	30/30	315	0.49	0.7	4.93× [3.26, 8.41] $9 \cdot 10^{-7}$	5.36× [3.57, 8.12] $6 \cdot 10^{-8}$
LP probing	30/30	25	0.44	0.5	0.55× [0.44, 0.70] $6 \cdot 10^{-5}$	5.36× [4.21, 6.34] 10^{-9}
uniform	30/30	228	0.14	0.3	3.86× [2.12, 8.05] $3 \cdot 10^{-4}$	1.55× [0.94, 2.72] 0.10
BP marginals (NSNet-based)	30/30	118	4.12	6.1	1.85× [1.12, 3.22] 0.04	50.13× [34.89, 64.65] 10^{-9}
Stage 1	30/30	116	0.14	0.2	1.88× [1.01, 2.93] 0.06	1.40× [1.24, 2.08] $5 \cdot 10^{-3}$
Stage 1+2	30/30	**50**	**0.08**	0.1	—	—

gation of step 2–3 discards a node because of that row, and a variable's score is the sum of (1 + weight) over the rows it appears in, with the rounded LP value first. *impact*: at the node, for every free variable and both values, the value is fixed and propagation run; the impact of the pair is 1 plus the number of variables propagation fixes, a contradictory pair is recorded as refuted (and its opposite fixed, as LP probing would), and the rule branches on the variable of largest summed impact, trying the value of smaller impact first. *LP probing*: for every free variable and both values the relaxation is re-solved with that value fixed; a value whose relaxation is infeasible is refuted and its opposite fixed; when nothing is refuted the rule falls back to the most-integral choice. *uniform*: a free variable and a value drawn uniformly. *oracle*: $\mathcal{S}(F, \mathbf{v})$ enumerated with CP-SAT at every node and rule (5) applied to the exact marginals. *BP*: damped sum-product (damping 0.9, tolerance 10^{-5}, at most 300 iterations) with exact cardinality-factor messages computed by prefix–suffix convolution, and rule (5) applied to the estimates.

Appendix D. Timing replication

Node counts are seed-invariant; Table D.11 replicates the wall-clock of the learned arms over two further evaluation seeds. The seed-0 runs of the two smaller sizes are 20–30% slower than seeds 1 and 2, which ran under a lighter concurrent load; the ratios in the main text use seed 0 throughout and are therefore conservative for the learned arms. At 28×80 the three seeds agree to within 1%.

Data and code availability

The instance generator, the training and evaluation code, the trained networks and the per-instance records behind every table will be made available in a public repository upon publication.

References

Aardal, K., Bixby, R.E., Hurkens, C.A.J., Lenstra, A.K., Smeltink, J.W., 2000a. Market split and basis reduction: Towards a solution of the Cornuéjols–Dawande instances. INFORMS Journal on Computing 12, 192–202.

Aardal, K., Hurkens, C.A.J., Lenstra, A.K., 2000b. Solving a system of linear Diophantine equations with lower and upper bounds on the variables. Mathematics of Operations Research 25, 427–442.

Achterberg, T., Koch, T., Martin, A., 2005. Branching rules revisited. Operations Research Letters 33, 42–54.

Amos, B., 2023. Tutorial on amortized optimization. Foundations and Trends in Machine Learning 16, 592–732.

Bestuzheva, K., Besançon, M., Chen, W.K., Chmiela, A., Donkiewicz, T., van Doornmalen, J., Eifler, L., Gaul, O., Gamrath, G., Gleixner, A., et al., 2021. The SCIP Optimization Suite 8.0. Technical Report 21-41. Zuse Institute Berlin.

Boussemart, F., Hemery, F., Lecoutre, C., Sais, L., 2004. Boosting systematic search by weighting constraints, in: Proceedings of the 16th European Conference on Artificial Intelligence (ECAI), pp. 146–150.

Cappart, Q., Moisan, T., Rousseau, L.M., Prémont-Schwarz, I., Cire, A.A., 2021. Combining reinforcement learning and constraint programming for combinatorial optimization, in: Proceedings of the AAAI Conference on Artificial Intelligence.

Chen, Z., Liu, J., Wang, X., Lu, J., Yin, W., 2023. On representing mixed-integer linear programs by graph neural networks, in: International Conference on Learning Representations.

Chu, G., Stuckey, P.J., 2015. Learning value heuristics for constraint programming, in: Integration of AI and OR Techniques in Constraint Programming (CPAIOR).

Cornuéjols, G., Dawande, M., 1999. A class of hard small 0-1 programs, in: Integer Programming and Combinatorial Optimization (IPCO), pp. 284–293.

Etheve, M., Alès, Z., Bissuel, C., Juan, O., Kedad-Sidhoum, S., 2020. Reinforcement learning for variable selection in a branch and bound algorithm, in: Integration of Constraint Programming, Artificial Intelligence, and Operations Research (CPAIOR), pp. 176–185.

Feng, S., Yang, Y., 2025. SORREL: Suboptimal-demonstration-guided reinforcement learning for learning to branch, in: Proceedings of the AAAI Conference on Artificial Intelligence.

Gasse, M., Chételat, D., Ferroni, N., Charlin, L., Lodi, A., 2019. Exact combinatorial optimization with graph convolutional neural networks, in: Advances in Neural Information Processing Systems.

Gershman, S.J., Goodman, N.D., 2014. Amortized inference in probabilistic reasoning, in: Proceedings of the Annual Meeting of the Cognitive Science Society.

Gurobi Optimization, LLC, 2024. Gurobi optimizer reference manual.

Han, Q., Yang, L., Chen, Q., Zhou, X., Zhang, D., Wang, A., Sun, R., Luo, X., 2023. A GNN-guided predict-and-search framework for mixed-integer linear programming, in: International Conference on Learning Representations.

Haralick, R.M., Elliott, G.L., 1980. Increasing tree search efficiency for constraint satisfaction problems. Artificial Intelligence 14, 263–313.

Hottung, A., Tanaka, S., Tierney, K., 2020. Deep learning assisted heuristic tree search for the container pre-marshalling problem. Computers & Operations Research 113, 104781. doi:10.1016/j.cor.2019.104781.

Khalil, E.B., Le Bodic, P., Song, L., Nemhauser, G., Dilkina, B., 2016. Learning to branch in mixed integer programming, in: Proceedings of the AAAI Conference on Artificial Intelligence.

Khalil, E.B., Morris, C., Lodi, A., 2022. MIP-GNN: A data-driven framework for guiding combinatorial solvers, in: Proceedings of the AAAI Conference on Artificial Intelligence.

Lenstra, A.K., Lenstra, H.W., Lovász, L., 1982. Factoring polynomials with rational coefficients. Mathematische Annalen 261, 515–534.

Li, Z., Guo, J., Si, X., 2023. G4SATBench: Benchmarking and advancing SAT solving with graph neural networks. Transactions on Machine Learning Research .

Li, Z., Si, X., 2022. NSNet: A general neural probabilistic framework for satisfiability problems, in: Advances in Neural Information Processing Systems.

Marques-Silva, J.a.P., Sakallah, K.A., 1999. GRASP: A search algorithm for propositional satisfiability. IEEE Transactions on Computers 48, 506–521.

Mazyavkina, N., Sviridov, S., Ivanov, S., Burnaev, E., 2021. Reinforcement learning for combinatorial optimization: A survey. Computers & Operations Research 134, 105400. doi:10.1016/j.cor.2021.105400.

Montanari, A., Ricci-Tersenghi, F., Semerjian, G., 2007. Solving constraint satisfaction problems through belief propagation-guided decimation, in: Allerton Conference on Communication, Control, and Computing.

Nair, V., Bartunov, S., Gimeno, F., von Glehn, I., Lichocki, P., Lobov, I., O'Donoghue, B., Sonnerat, N., Tjandraatmadja, C., Wang, P., Addanki, R., Hapuarachchi, T., Keck, T., Keeling, J., Kohli, P., Ktena, I., Li, Y., Vinyals, O., Zwols, Y., 2020. Solving mixed integer programs using neural networks. ArXiv:2012.13349.

Ohrimenko, O., Stuckey, P.J., Codish, M., 2009. Propagation via lazy clause generation. Constraints 14, 357–391.

Parsonson, C.W.F., Laterre, A., Barrett, T.D., 2023. Reinforcement learning for branch-and-bound optimisation using retrospective trajectories, in: Proceedings of the AAAI Conference on Artificial Intelligence.

Perron, L., Didier, F., 2023. OR-Tools CP-SAT. Google, version 9.x.

Pesant, G., Quimper, C.G., Zanarini, A., 2012. Counting-based search: Branching heuristics for constraint satisfaction problems. Journal of Artificial Intelligence Research 43, 173–210.

Pryor, J., Chinneck, J.W., 2011. Faster integer-feasibility in mixed-integer linear programs by branching to force change. Computers & Operations Research 38, 1143–1152. doi:10.1016/j.cor.2010.10.025.

Refalo, P., 2004. Impact-based search strategies for constraint programming, in: Principles and Practice of Constraint Programming (CP), pp. 557–571.

Scavuzzo, L., Chen, F.Y., Chételat, D., Gasse, M., Lodi, A., Yorke-Smith, N., Aardal, K., 2022. Learning to branch with tree MDPs, in: Advances in Neural Information Processing Systems.

Schnorr, C.P., Euchner, M., 1994. Lattice basis reduction: Improved practical algorithms and solving subset sum problems. Mathematical Programming 66, 181–199.

Selsam, D., Lamm, M., Bünz, B., Liang, P., de Moura, L., Dill, D.L., 2019. Learning a SAT solver from single-bit supervision, in: International Conference on Learning Representations.

Smit, I.G., Zhou, J., Reijnen, R., Wu, Y., Chen, J., Zhang, C., Bukhsh, Z., Zhang, Y., Nuijten, W., 2025. Graph neural networks for job shop scheduling problems: A survey. Computers & Operations Research 176, 106914. doi:10.1016/j.cor.2024.106914.

The FPLLL development team, 2023. fpylll, a Python wrapper for fplll. https://github.com/fplll/fpylll.

Vaezipoor, P., Lederman, G., Wu, Y., Maddison, C., Grosse, R.B., Seshia, S.A., Bacchus, F., 2021. Learning branching heuristics for propositional model counting, in: Proceedings of the AAAI Conference on Artificial Intelligence.

Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., Bengio, Y., 2018. Graph attention networks, in: International Conference on Learning Representations.

Wassermann, A., 2025. Solving the market split problem with lattice enumeration. ArXiv:2508.08702; to appear in Mathematical Programming Computation.

Williams, R.J., 1992. Simple statistical gradient-following algorithms for connectionist reinforcement learning. Machine Learning 8, 229–256.

Yan, F., Clautiaux, F., Froger, A., Albar, B., 2025. Generic machine-learning-augmented beam search for resource-constrained shortest path reformulations of combinatorial optimization problems. Computers & Operations Research 187, 107339. doi:10.1016/j.cor.2025.107339.

Table B.10: Refutation structure per rule (solved instances; medians over instances): nodes, decisions on the solution path, share of nodes in refuted subtrees, fraction of path decisions whose first value was refuted, mean cost of such a refutation, and the median depth of refuted decisions as a fraction of the path length.

rule	n	nodes	path	refuted share	first-child failure	refutation cost	depth frac.
21×60 (100)							
LP rule	100	7,762	41	0.995	0.111	1,677	0.42
most-fractional	100	662	40	0.940	0.135	121	0.27
maxSD	100	494	36	0.917	0.156	89	0.59
dom/wdeg	100	436	34	0.920	0.113	115	0.42
impact	100	1,464	31	0.973	0.412	98	0.54
LP-probing	100	130	17	0.809	0.125	44	0.80
uniform	100	2,112	40	0.981	0.432	129	0.56
Stage 1	100	628	46	0.923	0.064	246	0.52
Stage 1+2	100	325	28	0.908	**0.185**	**65**	0.62
S1 var / LP val	100	956	46	0.947	0.088	239	0.45
S2 var / LP val	100	292	25	0.909	0.160	63	0.37
LP var / S1 val	100	9,810	42	0.996	0.125	1,998	0.48
24×70 (100)							
LP rule	100	95,440	51	0.999	0.119	15,884	0.34
most-fractional	100	6,504	49	0.992	0.151	911	0.26
maxSD	100	5,637	44	0.991	0.163	835	0.53
dom/wdeg	100	5,263	42	0.992	0.141	876	0.38
impact	100	16,256	40	0.997	0.421	1,078	0.52
LP-probing	100	1,088	20	0.974	0.163	296	0.73
uniform	100	16,028	48	0.997	0.437	846	0.56
Stage 1	100	8,174	56	0.993	0.070	2,136	0.46
Stage 1+2	100	2,580	33	0.986	**0.206**	**387**	0.57
S1 var / LP val	100	6,998	55	0.992	0.103	1,319	0.45
S2 var / LP val	100	2,348	30	0.986	0.200	398	0.36
LP var / S1 val	100	69,872	50	0.999	0.134	9,734	0.45
28×80 (100)							
LP rule	40	1,029,039	59	1.000	0.126	137,471	0.28
most-fractional	99	187,381	58	1.000	0.173	21,053	0.21
maxSD	100	141,178	52	1.000	0.206	13,784	0.47
dom/wdeg	100	152,598	50	1.000	0.179	17,945	0.27
impact	49	149,624	49	1.000	0.432	7,945	0.49
LP-probing	78	17,148	22	0.998	0.222	3,594	0.60
uniform	91	520,855	57	1.000	0.477	20,198	0.51
Stage 1	90	212,271	65	1.000	0.079	41,686	0.41
Stage 1+2	100	58,026	39	0.999	**0.257**	**6,041**	0.49
S1 var / LP val	89	278,175	65	1.000	0.121	38,914	0.33
S2 var / LP val	100	69,665	37	0.999	0.269	7,026	0.33
LP var / S1 val	26	647,145	60	1.000	0.150	75,366	0.41

Table D.11: Timing replication of the learned arms over three evaluation seeds (seed 0 is the one reported in the main tables): instances solved and median wall-clock. Node counts are identical across seeds because the search is deterministic given the relaxation vertex.

size	arm	seed 0	seed 1	seed 2
21×60	Stage 1	100/100, 0.65 s	100/100, 0.48 s	100/100, 0.49 s
21×60	Stage 1+2	100/100, 0.36 s	100/100, 0.27 s	100/100, 0.26 s
24×70	Stage 1	100/100, 6.60 s	100/100, 6.50 s	100/100, 6.42 s
24×70	Stage 1+2	100/100, 2.39 s	100/100, 1.94 s	100/100, 1.93 s
28×80	Stage 1	90/100, 182.65 s	90/100, 182.50 s	90/100, 182.38 s
28×80	Stage 1+2	100/100, 46.61 s	100/100, 46.30 s	100/100, 46.34 s